FREE RESOURCE · ARCHITECTURE

The Multi-Tenant SaaS Architecture *Checklist.*

The decisions you can't cheaply undo six months later.

By Ahmed Farid — Senior Software Engineer · iamahmedfarid.com

Most multi-tenant SaaS rewrites don't happen because the code was bad. They happen because a boundary was drawn wrong on day one — and by the time it hurts, it's load-bearing. This is the checklist I run before writing a single screen, distilled from shipping multi-tenant platforms across the Gulf, US & UK. Use it as a pre-build gate: if you can't answer a question, that's the next conversation to have — not the next feature to build.

01 Tenancy model — decide before anything else

- [ ] **Isolation strategy chosen and written down:** shared schema (tenant_id), schema-per-tenant, or database-per-tenant — each has a different blast radius and cost curve.
- [ ] **Tenant resolution path defined:** subdomain, path, header, or token claim — resolved in middleware, not controllers.
- [ ] **Every query is tenant-scoped by default**, not by developer discipline. A global scope that fails **closed** (no tenant = no data).
- [ ] **The "noisy neighbor" question answered:** can one tenant's load degrade another's? What's the ceiling?
- [ ] **Cross-tenant admin access is a separate, audited path** — never the same code path as tenant users.

02 Identity, roles & access

- [ ] **Roles modeled as first-class concepts**, not booleans that multiply.
- [ ] **Separate auth guards** for distinct actor types (admin / company / customer) rather than one overloaded user table.
- [ ] **Permission checks centralized** in a policy layer, not sprinkled in views.
- [ ] **Tenant + role together** decide access — a valid user in the wrong tenant sees nothing.
- [ ] **Impersonation / support access** is logged, time-boxed, and reversible.

03 Data & billing awareness

- [ ] **Billing is a first-class concept in the data model** from day one — plans, limits, usage, and the seams to meter them.
- [ ] **Usage-limiting is enforceable** (seats, API calls, storage) without a schema migration later.
- [ ] **Schema first, indexes second** — but the tenant_id composite indexes that matter at scale are identified now.
- [ ] **Soft-delete & data-retention policy** decided per entity.
- [ ] **Tenant data export & deletion** possible (GDPR/offboarding) without a manual DBA operation.

04 Real-time & concurrency (if applicable)

- [ ] **The server is the single source of truth** for ordering — never the client.
- [ ] **Atomic operations** for contended state (Redis atomic increments for counters, bids, inventory).
- [ ] **A push layer** (WebSocket/SSE) mirrors state to watchers — with a latency target you actually measure.
- [ ] **Idempotency keys** on anything retryable (payments, webhooks).

- [ ] **Backpressure & reconnection** handled — what happens when 200 clients reconnect at once?

05 Integrations & money

- [ ] **Payment gateway(s) chosen per region** and abstracted behind one interface so a second gateway isn't a rewrite.
- [ ] **Webhooks verified, idempotent, and queued** — never processed inline on the request thread.
- [ ] **ERP/CRM sync direction defined** (one-way vs bidirectional) with conflict resolution decided before the first sync.
- [ ] **External calls have timeouts, retries, and a circuit breaker** — a slow third party can't take you down.

06 Delivery & operations

- [ ] **Deploy topology decided:** environments (alpha/beta/prod) and a pipeline that survives a Friday deploy.
- [ ] **Migrations safe to run by someone who isn't you, at 2 a.m.** (reversible, non-locking where it counts).
- [ ] **Observability from v1:** structured logs, error tracking, and the 3–5 metrics that signal health.
- [ ] **Feature flags** so vertical slices ship early and dark.
- [ ] **A runbook exists** — the handoff is part of the work, not an afterthought.

07 The "can't cheaply undo" list

Get these right on day one. Everything else you can refactor.

1. Tenant isolation strategy
2. Auth / guard structure
3. Billing in the data model
4. Server-authoritative real-time ordering
5. API contract & versioning
6. Deployment topology

Want a second pair of eyes on yours?

I do architecture reviews and fixed-scope builds for real-time, multi-tenant SaaS. A 30-minute scoping call is free — and often saves a six-month rewrite.

Book a call → **Connect on LinkedIn**